

1 Lesson 22-09-26

1.1 Introduction

The aim of this course is to provide basic knowledge about the architecture of modern *microprocessor*-based systems. Special emphasis will be put on the *microprocessor core* itself.

The course is structured in two main blocks:

1. Modern processor architecture. This will cover computer design, the *RISC* processor architecture, superscalar processors and disrupting architectures;
2. *ARM*-based systems, that is *ARM* assembly language and *ARM Cortex-M3* based systems. Both development boards and emulation systems will be used to aid in the study of *ARM*-based systems.

1.2 CISC vs. RISC

We first go over what an **ISA** (*Instruction Set Architecture*) is in the context of processor design. Then we distinguish between **CISC** (*Complex Instruction Set Computer*) and **RISC** (*Reduced Instruction Set Computer*) architectures.

1.2.1 High level vs. Low level languages

Many programming languages exist. They can be classified into:

- **High-level** programming languages, which abstract over the machine to offer more sophisticated capabilities. This gives developers the freedom to express the same algorithm in many ways, and usually to express the same algorithm with significantly less lines of code than those used in low-level languages;
- **Low-level** programming languages, which stay closer to an **ISA** (*Instruction Set Architecture*). An ISA is a specification which defines:
 - Supported instructions;
 - Data types;
 - Registers;
 - Addressing modes.

In low-level languages, operations are smaller than those provided in high-level languages, so even simple algorithms usually take more instructions to express.

We can express this difference by taking the simple example of a loop:

1. In high-level languages such as *Python* and *C*, loops are expressed via a simple and predefined syntax which allows to compactly express iteration value initialization, update and the exit condition;
2. In a low-level language, all of these operations are distinct: the programmer has to explicitly initialize an iteration value, perform a comparison to check for the exit condition, and manually update the iteration value.

Another idea to note is that high-level languages are usually designed to be *portable*, that is *interpreted* or *compiled* for a variety of systems. On the other hand, low-level

languages are usually designed for a single machine (a single type of processor), and porting to other platforms requires manual conversion work.

Programs written in high-level languages are usually compiled on the machines they are meant to run on: this means they are converted to low-level machine code which can be directly executed. For environments where running a full compiler is not feasible (such as a development board), a technique known as *cross-compilation* can be used: the program is compiled on a second machine, into machine code for the first machine. All of this of course doesn't apply to low-level work, where we are expected to write the same routines from scratch, and likely differently, for each target architecture.

1.2.2 Data types and addressing

An ISA also defines the **data types** the CPU can address. For instance, the RISC architecture might allow a programmer to address bytes, half words (double bytes) and words (double half words). With this being said, we note that usually low-level languages interpret all data as *binary* data (complex types such as *integer*, *float*, ... are usually added in as high-level constructs).

There are two main methods of performing addressing:

- **Register-Memory** architectures: operands of instructions can be immediates (contained within the instruction encoding), or contained in memory *or* in registers. If *all* operands can be in registers or in memory (or in combinations), the architecture is called *register-plus-memory*. Clearly, as memory accesses are also operations that have to be carried out by the processor, these kinds of ISAs are more complex.
- **Register-Register** architectures: these are also called **load-store** architectures. Instructions of register-register architectures are explicitly divided into:
 - Memory access (*load* and *store* between memory and registers);
 - **ALU** (*Arithmetic and Logic Unit*) operations (only between registers). The ALU itself is one of the most important components of a processor.

It follows that operands of ALU instructions are immediates or contained in registers.

In CISC (usually *register-to-memory*) instruction sets, the order of instructions can affect the number of memory accesses. The more the number of memory accesses increases, the more performance degrades. Load-store architectures, by their nature, do not suffer from this problem

1.2.3 CISC

CISC (*Complex Instruction Set Computer*) was the name given to the first computer designs. The main purpose is to complete a task in as few assembly lines as possible. This makes sense in an environment where programmers themselves are tasked with writing assembly code, as we said earlier that low-level languages tend to express the same concepts with more lines of code (we refer to this as a *semantic gap* between low and high-level languages).

Common examples of CISC CPUs are:

- x86 (1978) and x64 (2003) family, by Intel and AMD;
- Motorola 68000 series (1979);
- IBM System/360 family (1964);
- PDP-11 (1970) and VAX families (1977), by DEC.

Some characteristics of CISC are:

- Instruction encoding has *variable length*: for an immediate instruction we need to store the immediate, whereas for register to register or register to memory instructions we need to store register and memory addresses. As an example, x64 family instructions are between 1 and 6 bytes;
- The processor architecture has to be more *complex*, as a single instruction can express complex operations;
- The number of *clock cycles* required for execution of a single instruction is wildly variable: if an instruction requires a memory access, we need to wait for the memory to respond, which may happen on timescales far larger than those of the CPU;
- The resulting *code size* is far smaller than that of other architectures.

1.2.4 RISC

RISC (*Reduced Instruction Set Computer*) is the name given to a family of processors initially developed during the 1980s. The main purpose is to make the processor as simple as possible: this requires more work on the programmers' side, but allows the CPU to move much faster and execute many more instructions in the same amount of time as a CISC CPU. The limitation of more complex code, then, can be removed by making use of highly optimized *compiler* which take the toll of manual assembly programming away from programmers.

Common examples of RISC CPUs are:

- ARM family (1985), designed by ARM Holdings, which will also be the focus of the second part of this course;
- MIPS family (1984), by MIPS Technologies Inc.;
- RISC-V (2014), an example of a *register-to-register* ISA;
- SPARC family (1985), by Sun Microsystems;
- PowerPC (1991), by Apple, IBM, and Motorola.

Some characteristics of RISC are:

- The instructions themselves are *simpler*;
- The length of the instructions can be *fixed* (for example, in RISC-V they have a fixed size of 32 bits). A fixed instruction size sometimes requires some bits of the instruction encoding to remain unused: this is still preferred over variable length encoding as it allows the component of the CPU that performs the *fetch-decode* cycle to be far simpler;
- The processor architecture is *simpler*;
- The number of clock cycles required to execute an instruction is normally *lower* than in CISC, and there is less overall variance. This makes it easier to design and validate both RISC code and processors, as the amount of time taken by each instruction is easy to estimate (optimistically, 1 cycle per instruction).

Common factors that disallow having exactly 1 cycle per instruction involve:

- *Latencies* introduced by CPU optimizations, such as pipeline flushes and branch misses;
- *Memory accesses*: load and store instructions always have to at least wait for the cache, and if a miss happens, for the memory, which we said is usually slower than the CPU;

- Some *complex instructions* can be introduced back into the architecture: these give the programmer more power, but also require more clock cycles to complete.
- The resulting *code size* is larger than that of other architectures.

All of these characteristics contributed to making RISC architectures more popular than CISC architectures nowadays. In fact, some modern CISC processors actually convert the CISC instructions to an internal RISC representation, which then gets actually executed on the processor.

Also, the tradeoff given by longer code length is balanced by the fact that we are allowed to have *easier* CPUs, that is CPUs that are easier to *design* and *validate*. Some extensions have been introduced (such as the C "Compressed" extension for RISC-V) to make RISC-V instructions smaller and reduce the code memory footprint.

1.2.5 Comparison of addressing modes

As an example, let's compare the available addressing modes in x86 (a CISC architecture) and RISC-V (a RISC architecture). We then go over a specific detail we will encounter in ARM architectures.

- In **x86**, a number of addressing modes are available:
 - Register mode: `MOV AX, BX`
 - Immediate mode: `MOV AX, 2000`
 - Displacement or direct mode: `MOV AX, [DISP]`
 - Register indirect mode: `MOV AX, [DI]`
 - Based indexed mode: `MOV AX, [BX + DI]`
 - Indexed mode: `MOV AX, [SI + 2000]`
 - Based mode: `MOV AX, [BP + 0100]`
 - Based indexed displacement mode: `MOV AX, [SI + BP + 2000]`

Such a large amount of modes could look a bit confusing. The idea is that modern x86 processors support **SIB** (*Scale-Index-Base*) addressing for arrays, *Mod R/M* bits, which allow for the following parameters:

- **Base** is a general-purpose register, containing a base memory address;
- **Index** is a general-purpose register containing a memory offset;
- **Scale** is a special constant, which can be 1, 2, 4, or 8;
- **Displacement** is an arbitrary constant, offered as an immediate in the instruction.

From these parameters, the processor computes an *Expected Address (EA)* is generally defined as:

$$EA = \text{Base} + \text{Index} \times \text{Scale} + \text{Displacement}$$

Any of these components can be omitted, resulting in the complex array of addressing modes shown above (the indexed/based move differ mainly in semantics because of historical reasons).

In the ISA, these addressing modes are specified by the instruction encoding, by the *ModR/M* byte, with an optional *SIB* byte. For more information, refer to the notes available at this very long URL: <https://raw.githubusercontent.com/seggiani-luca/appunti-rl/34228f66db395637bd1824d04f3130b977cc0ce4/master/master.pdf> (in italian).

We show one last table to summarize:

Addressing mode	Example	Loaded address
Register	ADD AX, BX	AX
Immediate	ADD AX, FFh	FFh
Displacement	ADD AX, [FFh]	mem[FFh]
Based	ADD AX, [BX]	mem[BX]
Indexed	ADD AX, [CX]	mem[CX]
Scale indexed	ADD AX, [CX*4]	mem[CX × 4]
Indexed based	ADD AX, [BX + CX]	mem[BX + CX]
Scale indexed based	ADD AX, [BX + CX*4]	mem[BX + CX × 4]

- In **RISC-V**, the same operations is significantly simpler. We have available:
 - ALU operations:
 - * Register addressing mode: **ADD** t0, t1, t2
 - * Immediate addressing mode: **ADDI** t0, t1, 10

with the first register taken as the destination, in common RISC fashion.
 - Load / store operations, in base + displacement addressing mode: **LW** t2, 4(t1), where the constant before the open parenthesis is an immediate offset, and the register contains a memory address. Of course, since for instant RV32x is a 32 bit architecture, 32 bit instructions cannot fully contain immediate addresses: this is taken into account by special facilities in the processor which allow full address loading via multiple instructions.
- The same is available in ARM, with the distinction of having *pre-indexed* and *post-indexed* memory operations, that is control over the time when the base register is updated.

Being more specific, ARM allows load and store instructions to optionally *alter* the address contained within a base register. 3 forms are available:

 - **ldr** r0, [r1, #4] is equivalent to immediate offset addressing in RISC-V, with #4 being the immediate operand;
 - **ldr** r0, [r1, #4]!, with the additional !, means performing a *write-back* or **pre-indexing** operation. The calculated address, obtained by adding the offset, is *written back* into the base register before the load operation. The result we get is that of a load from the base address plus the offset, with the base address reflecting the change after execution. Compared to RISC-V, this saves an instruction;
 - **ldr** r0, [r1], #4 means performing a **post-indexing** operation. This means increasing the value of the base register by offset, after performing the load. We can think of this as being analogous to the ++ syntax available in C. This also saves an instruction compared to RISC-V.

As we can see, the main point of this complication is to save a few instructions when accessing arrays, or generally where striding memory access patterns are needed in ARM architectures.

2 Lesson 23-09-26

2.1 Processor design

The first general-purpose computer was created in the late 50s. A Personal Computer, that can now be bought for about \$500, is practically equivalent (in terms of performance and memory) to what could be bought for about \$1M in 1985.

This evolution has been made possible by:

- Advances in semiconductor technology
- Innovations in computer architecture
- Advances in software.

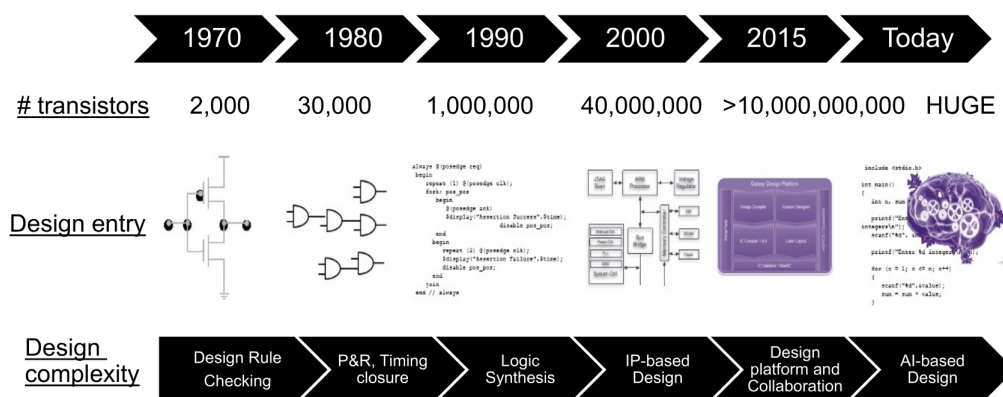
Despite this the main architecture used today is still the same that was first applied to real computers, that is the *Von Neumann Architecture*.

2.1.1 Advanced architectures

Of course, there are modern advancements which have to be noted. We refer to these as *advanced architectures*:

- **Advances in computer architecture:** performance has been improved without fundamentally changing the Von Neumann model through techniques such as pipelining, which overlaps the execution of multiple instructions, and superscalar execution, which allows multiple instructions to be executed in parallel in a single clock cycle. Other major improvements include caches, branch prediction, and out-of-order execution.
- **Advances in software and tooling:** compilers have become increasingly sophisticated at optimizing programs for the underlying hardware, exploiting instruction-level parallelism, registers, caches, and specialized instructions. Better operating systems, programming languages, debuggers, profilers, and development tools have also made increasingly complex hardware practical to use.

Chip-design history



2.1.2 Moore's law

Definition 2.1: Moore's law

The number of devices (i.e., transistors) that can be integrated into a single chip doubles every 18/24 months.

Despite the name, Moore's law describes an empirical relationship, not a scientific law. This type of observation, an experience curve effect, quantifies efficiency gains from learned experience in production. In fact, Moore's empirical evidence did not imply that the historical trend would continue; nevertheless, his prediction has held since 1975 and has since become known as a law.

We can also say that thanks to Moore's law, we are today able to integrate more and more transistors within one circuit, giving many powerful options to designers.

Many technologies are available to streamline chip design (for instance, *synthesis* and automated tools). Having more transistors within a single processor also allows for advanced features. For instance, something like a *floating point* processing unit executing alongside the main processor takes a large number of transistors to implement, which larger chip transistor densities allow.

2.1.3 Processor performance evolution

Processor **performance** can be measured by counting the number of operations executed or the time needed to execute them.

- User point of view: performance = response time (time between start and completion of an operation);
- System manager point of view: performance = *throughput* (total amount of work done in the time unit).

Processor performance evaluation is often performed by letting the computer to execute applications and observing its behavior. Unfortunately, the choice of the applications severely affects the performance. In the ideal case, one should use as workload the mix of applications the user will run. However, they are normally unknown, and largely variable from one user to another. Therefore, some **benchmarks** are selected to mimic real cases.

Processor performance has steadily increased since the 1970s, where the first main-frame machines started being ranked based on performance.

- From the 1970s to around the 1980s, yearly performance increases were of about 25% ~ 30%. We have to recall that this was a time when mainframes and mini-computers dominated the market: performance improvements were driven mainly by advances in integrated-circuit technology, together with architectural improvements such as larger register sets, more buses, caches, and pipelining.
- In the 1980s, the introduction of **RISC** (*Reduced Instruction Set Computer*) architectures, as opposite to **CISC**, led to a yearly performance gain of around 50%. These allowed for major processor redesigns, which enabled optimization unfeasible (at least at the time) on CISC architectures;
- In the first years of the 2000s, processor performance growth hit a slow down because aiming for higher and higher clock speeds became unfeasible. There were 3 main reasons:

- **Power** issues: higher clock speeds meant higher power dissipation, which cooling could only partially keep under control;
- Lower **ILP** (*Instruction Level Parallelism*): this was disadvantageous for the *superscalar* architectures used at the time, characterized by high pipeline depth and instruction level parallelism;
- Unchanged **memory latency** remained a big bottleneck.
- The solution most vendors adopted from the mid 2000s onwards, and which to this day is widely adopted, is *multi-core* processors. In fact, since 2004, major industries stopped the race for higher-performance single processor projects running at higher clock frequencies, and embraced *multi-core* CPUs.
In a **multi-core** processor, we effectively have multiple processors (or processor *cores*) running in parallel on a single, or multiple tasks. Note that multi-core setups increase *throughput*, and not *response time*.
- In more recent times (mid 2010s) we have seen a 12% year improvement, or doubling every 8 years (mainly due to the limits on instruction parallelism). This has slowed down to a 3.5% performance improvement, or doubling every 20 years, in the last 10 years (at the time of writing).

Today, we can not only talk about processor improvements, but the fact that modern computer systems employ various types of *accelerators*, that is circuits intended to support specific applications alongside the CPU, like graphics or AI (an example is given by **NPU**s, *Neural Processing Units*).

In the recent years, the widespread adoption of Artificial Intelligence (AI) applications prompted the introduction of new computing architectures denoted as AI accelerators, such as **GPU**s (*Graphics Processing Units*). GPUs were originally introduced for graphics applications, but their current success is due to their ability to effectively manage large and regular data structures. The GPU market (in 2024 corresponding to about 18 B\$), is expected to grow to 406 B\$ by 2033 (with a Compound Average Growth Rate, or CAGR, of 41,39%). The CAGR of the CPU market is about 7%.

2.1.4 Costs

In processor, and more generally computer design, **costs** have to be taken into account. These can be divided into two main categories:

- **Design costs:** these are largely *fixed* or *non-recurring* costs, usually referred to as **NRE**s (*Non-Recurring Engineering costs*). They include activities such as architecture design, circuit design, verification, physical design, software development, and mask production. NRE costs grow rapidly with chip complexity and with the sophistication of the manufacturing process. In particular, moving to a newer process node requires expensive new design tools, masks, **IP** (*Intellectual Property*), and extensive verification. Since these costs are paid before mass production, they have to be amortized over the number of chips sold. Therefore, a high-volume product can tolerate large NRE costs because they are distributed over many units, whereas for a low-volume product they can represent a significant portion of the final cost;
- **Manufacturing costs:** these are the *recurring* costs associated with producing each chip. They include wafer fabrication, packaging, assembly, and testing. The semiconductor manufacturing process itself requires extremely expensive fabrication facilities (*fabs*), whose cost increases substantially for advanced process nodes.

An important factor when evaluating manufacturing costs is the **yield**, i.e., the percentage of manufactured chips that satisfy all specifications and pass the testing phase. Not every die on a wafer is functional: manufacturing defects can make some chips unusable. Consequently, a low yield increases the effective cost of every working chip. Yield normally improves over time as the manufacturing process matures. This phenomenon is known as the **learning curve**: defects are progressively reduced, manufacturing parameters are optimized, and the production process becomes more reliable. As yield increases, the effective cost per functional chip decreases.

Testing and validation also contribute significantly to the recurring cost of a processor. Complex processors require extensive electrical testing and, in many cases, additional screening and characterization before they can be sold. Thus, manufacturing cost is not simply the cost of fabricating the silicon itself, but includes all the steps required to obtain a tested and sellable device.

2.1.5 The computer market

The computer *market* is currently split in 5 different areas:

- *Personal Mobile Devices (PMDs)*: this area includes smartphones and tablet computers. Emphasis is on energy efficiency and real-time applications. Here, system price is around \$100 - \$1,000, with the microprocessor price at \$10 - \$100. Architectures are usually highly parallel (and in fact smartphones were the earliest devices to make heavy use of multi-core processors);
- Desktop computing: this area covers from PCs to workstations (and in some sense, even laptops). The main target of this area is to optimize the price-performance ratio. The system price is around \$300 - \$2,500, and of course the microprocessor price follows \$50 - \$500. Power is much less important than before, as we don't use batteries but power off the grid;
- Servers: these systems provide larger-scale and more reliable computing services. The main parameters of products in this area are:
 - Availability;
 - Scalability;
 - Throughput.

System price is around \$5,000 - \$10,000,000, with microprocessor price around \$200 - \$2,000;

- *Clusters / Warehouse Scale Computers (WSC)*: systems that offer "Software as a Service (SaaS)" or "Platform as a Service (PaaS)". The emphasis is on availability, price-performance, and power consumption. *Supercomputers* make up a part of this category, where the emphasis is on floating-point performance and fast internal networks. The system price is high at around \$100,000 - \$200,000,000, with the microprocessor price around \$50 - \$250 (because at times we might use off the shelf processors in massively parallel setups);
- Embedded computers: this area is a major application area of the computer market. It covers all special-purpose computer-based applications (from microwaves to coffee machines, from automotive to videogames). Adopted microprocessors vary from cheap low-end 8-bit processors to very efficient (and expensive) high-end processors. The system price ranges from \$10 - \$100,000, with microprocessor price at around \$0.01 - \$100 (possibly extremely cheap).

Special requirements often existing in this area are:

- Real-time performance requirements;
- Memory minimization;
- Power consumption minimization;
- Reliability constraints.

Embedded problems are often solved resorting to one of the following "standard" solutions:

- Standard processor, custom logic, custom software. We call the combination of standard processors with custom logic **ASIC** (*Application Specific Integrated Circuit*);
- Standard processor, custom software;
- Standard DSP, custom software.

Programmable devices (such as **FPGAs**) are playing a growing role.

2.1.6 Power

Continuous increase in system complexity and device integration often leads to problems with **power consumption**. Power consumption may be critical under two aspects:

- **Power**: *static* power, consumed when a device is just running without doing work, and *dynamic* power, which mainly depends on the frequency devices run at (and also the square power). Until now it has been dominated by dynamic power, i.e., that consumed by each transistor when switching between different states.
- The **dynamic** power for each transistor can be evaluated by the usual formula:

$$P_{dynamic} = \frac{1}{2} f V^2 C_{eq}$$

- Whereas **static** power can be evaluated by:

$$P_{static} = VI$$

and amounts around 50% of power consumption.

For this very reason, voltage continuously dropped in the last years.

- **Energy** consumed: it is given by:

$$E = C_{eq} V^2$$

It is mainly of interest for *portable* devices, where we are draining a battery (these include mobile phones and laptops, like the one the author of this notes is writing on, which will likely shut down if a power outlet is not found stat).

2.1.7 Dependability

Dependability is the quality of the system to deliver a correct service. Dependability of computer systems is traditionally very high, but it can be lowered by:

- Bugs in the design of the hardware;
- Bugs in the software;
- Defects in the hardware (introduced by the manufacturing process);

- Faults happening during the product operation.

Many **safety critical** systems exist. In the past, possible misbehaviors of computer systems were critical in some areas, such as

- Space;
- Avionics;
- Nuclear plants control.

In more recent years, computer-based systems expanded to other safety-critical areas, such as:

- Rail-road traffic control
- Automotive
- Biomedical
- Telecommunications.

Metrics for dependability are defined as:

- *Mean Time To Failure (MTTF)* or *Failures In Time (FIT)*, which is the reciprocal. 1 FIT equals 1 failure in one billion hours;
- *Mean Time To Repair (MTTR)*;
- *Mean Time Between Failures (MTBF)*.

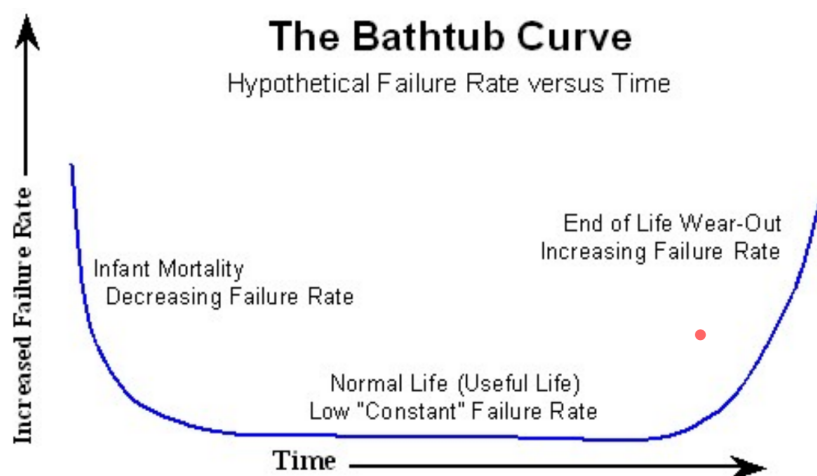
The three measures are related by the formula:

$$MTBF = MTTF + MTTR$$

Finally, availability is the probability that a system works correctly in a generic time instant (usually measured in FITs).

The **bathub curve** describes the probability of faults given the age of a component. We give it as a final example of how to take dependability into account.

- The *infant mortality* zone is where faults of components show themselves early. Controlled *burn-in* can partially solve this problem;
- The *normal life* zone is where expect to spend time using devices;
- The *end of life* zone is where failures hidden in components which otherwise worked fine start appearing. This is of particular interest to data centers where components are heavily stressed.



2.1.8 Reproducibility

Going back to the topic of *performance*, we introduce **reproducibility**. Information about execution times on benchmarks should allow reproducibility. This means reporting detailed information about:

- Hardware (system configuration);
- Software (OS, compiler, program);
- Program input.

A common issue is comparing and summarizing performance:

- **Problem 1:** I know the performance of one machine on a set of programs: which is its global performance?
- **Problem 2:** I know the performance of two machines on the same set of programs: which is their relative performance?

A number of metrics have been proposed, such as total execution time, arithmetic mean and weighted means.

2.1.9 Ahmdal's law

Definition 2.2: Ahmdal's law

We define the *speedup* from a given architecture enhancement as:

$$\text{speedup} = \frac{\text{performance with enhancement}}{\text{performance without enhancement}}$$

The speedup resulting from an enhancement depends on two factors:

- $\text{fraction}_{\text{enhanced}}$: the fraction of the computation time that takes advantage of the enhancement;
- $\text{speedup}_{\text{enhanced}}$: the size of the enhancement on the parts it affects.

We can compute the execution time after the speedup as:

$$\text{execution time}_{\text{new}} = \text{execution time}_{\text{old}} \cdot ()$$

Applying the definition of speedup, we get:

$$\text{speedup} = \frac{\text{performance with enhancement}}{\text{performance without enhancement}} = \frac{1}{(1 - \text{fraction}_{\text{enhanced}}) + \frac{\text{fraction}_{\text{enhanced}}}{\text{speedup}_{\text{enhanced}}}}$$

Ahmdal's law is particularly useful in the study of parallel architectures, as it expresses the speedup given by optimizing parallelizable parts of computing tasks. More specifically, we use it to explain why we can never have $S = N$ with N processor cores.

We can say that there will always be intervals of sequential execution (T_{seq}), spent executing instructions that cannot be parallelized, such as I/O operations, conditional statements, inherently sequential algorithms, etc... Then the speedup S can be defined as:

$$S = \frac{T_1}{T_{\text{seq}} + \frac{T_1 - T_{\text{seq}}}{N}}$$

with T_1 being the full execution time, i.e., the speed-up gain comes only from the parallelizable parts of the program ($T_1 - T_{\text{seq}}$), while the sequential parts (T_{seq}) cannot benefit significantly from parallelism.

The problem is that, taking the limit for $N \rightarrow \infty$, we obtain:

$$\lim_{N \rightarrow \infty} S = \lim_{N \rightarrow \infty} \frac{T_1}{T_{\text{seq}} + \frac{T_1 - T_{\text{seq}}}{N}} = \frac{T_1}{T_{\text{seq}}}$$

Thus, as N increases, the speed-up becomes dominated by the sequential parts: the parallelizable portions are progressively reduced, while the sequential portions remain and ultimately become the limiting factor for the overall system performance.

2.1.10 The CPU performance equation

Let's talk about measuring the time required to execute a program. There are several possible approaches:

- By observing the real system;
- By simulation;
- By applying the **CPU performance equation**.

Definition 2.3: CPU performance equation

CPU execution time is given by:

$$\text{CPU Time} = \left(\sum_i \text{CPI}_i \times \text{IC}_i \right) \times T_C$$

where:

- CPI_i is the number of clock cycles required by instruction i ;
- IC_i is the number of times instruction i is executed in the program;
- T_C is the clock period.

This equation is quite basic, as it doesn't take any processor optimizations into account. Still, it is a feasible metric for processor performance estimation, before the processor is even actually implemented.

2.2 Instruction Set Principles

Every processor owns an *Instruction Set Architecture (ISA)*. The ISA provides the information about the processor required by programmers and compilers to produce correctly working code:

- The set of available *instructions*;
- Instruction *encodings*;
- Available *registers*;
- etc...

Different processors may share the same ISA (e.g., Intel and AMD processors, or processors of the same family).

The Instruction Set Architecture (ISA) is basically how the computer is seen by the programmer or the compiler. There are many alternatives possible for the ISA designer,

any choice modifies the way to encode the instructions. The different alternatives may be evaluated in terms of:

- Processor performance;
- Processor complexity;
- Compiler complexity;
- Code size;
- Power consumption;
- etc...

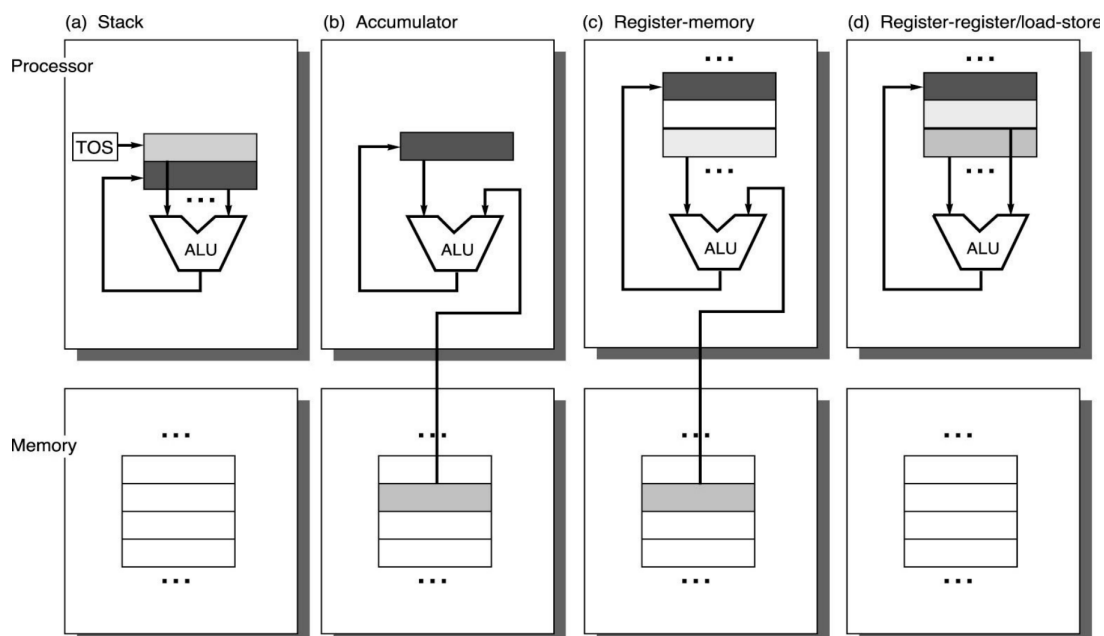
Different product areas may assign different weights to the above parameters.

An ISA specification is particularly important because it is usually the only set of information released by a company about a processor. Of course, the internal implementation details are trade secrets, but the ISA specification is made available in order to allow programmers to develop for a certain platform.

2.2.1 CPU taxonomies

CPUs (and their ISAs) are often classified according to the type of their internal storage:

- Stack;
- Accumulator;
- Registers:
 - Register-memory, as in x86;
 - Register-register (load-store). Basically all RISC architectures are of this kind;
 - Memory-memory (no real cases).



Currently, all processors are *General-Purpose Register (GPR)* machines. The reason for this is:

- Registers are faster than memory;
- Registers are easier for a compiler to use (or better, to *optimize*).

CPUs can also be classified according to:

- Typical number of operands per ALU instruction (2 or 3). In RISC-V and ARM, this is almost always 2;
- Typical number of memory operands per ALU instruction (from 1 to 3, for architectures that feature memory operands in the first place in instructions rather than load/store).

2.2.2 Memory addressing

We have some orthogonal characteristics relating to memory *addressing*:

- **Little Endian** vs. **Big Endian**: processors usually address memory in bytes, organized into words. The order of the bytes within a word matters:
 - *Little Endian* puts the byte with the lowest address (X...X000) at the least significant position. The address of the data is that of the least significant byte;
 - *Big Endian* puts the byte with the lowest address (X...X000) at the most significant position. The address of the data is that of the most significant byte.
- **Aligned** vs. **misaligned** accesses:
 - *Aligned* means allowing only aligned accesses not straddling word (usually 2 byte) bounds. It is a limitation;
 - *Disaligned* access, on the other hand, requires: hardware overhead and performance overhead.

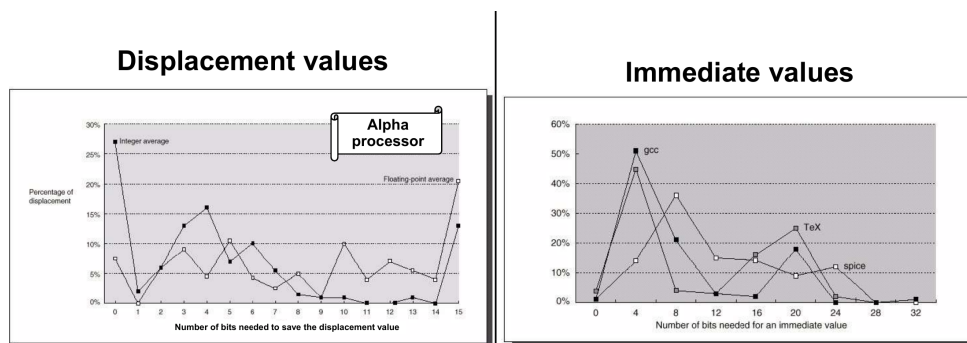
We shall also note that in GPR machines an addressing mode specifies a constant, a register, or a memory location (through its effective address).

We remember the addressing modes noted in section 1.2.5 for x86. Some of these are shared with ARM and RISC-V RISC architectures (mainly register mode, with destination register, immediate and displacement modes for load/store operations).

By carefully choosing the addressing modes, one can obtain some important consequences:

- Reducing the number of instructions;
- Increasing the CPU architecture complexity;
- Increasing the average *Cycles Per Instruction (CPI)*.

An interesting thing to note is the size devoted to immediate operands in immediate instructions. That is, how large can the immediate value be, when it is embedded in the addressing part of the instruction? Note that memory displacements basically are a special case of immediate operands, and as such fall in the same category and have the same problem. In fact, we had already noted this issue in section 1.2.5.



An experimental evaluation can be performed. From the plots, compiled from a DEC alpha processor for displacement and for typical compiled applications for immediates, we get that:

- Displacement, immediate and register indirect modes represent from 75% to 99% of the addressing modes;
- The address size for displacement mode should be from 12 to 16 bits (75% to 99% of the displacements);
- The size of the immediate field should be at least 8 or 16 bits (50% and 80% of the cases, respectively).

RISC-V uses the solution of doing 12 bits immediates. As we previously stated, bigger immediates have to be transferred to register in multiple instructions. This tradeoff is made valid by the simplicity gains of the processor architecture.

2.3 Operations in the instruction set

Clearly, a key component of an ISA is the **instruction set** made available to the programmer. Different categories of instructions exist:

Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, and, subtract, or
Data transfer	Loads-stores (move instructions on machines with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management instructions
Floating point	Floating-point operations: add, multiply
Decimal	Decimal add, decimal multiply, decimal-to-character conversions
String	String move, string compare, string search
Graphics	Pixel operations, compression/decompression operations

We should be making the common case fast: not all the instructions are executed with the same frequency! When designing and implementing an instruction set, the most commonly executed instructions should be made faster than others.